

Testing of Web Applications and Web Services

Meenakshi D'Souza

International Institute of Information Technology Bangalore.

Overview

- Introduction to relevant aspects of web applications.
- Issues in testing of web applications.
- Testing static hyper text web sites.
- Testing dynamic web applications.
  - Client-side testing of web applications.
  - Server-side testing of web applications.

NPTEL

Web application

- A web application is a program that is deployed on the web.
  - Uses HTML as the user interface.
  - Web-deployment means they are available worldwide.
  - They accept requests through HTTP and return responses.
- It is like a client-server application where the client runs in a web browser.
- Examples: web mail, online retail sales, wikis etc.

NPTEL

Web services

- A web service is a web-deployed program that accepts XML messages wrapped in SOAP.
  - Usually no user interface with humans.
  - Service must be published so other services and applications can discover them.

NPTEL

General characteristics of web software

- Composed of independent, loosely coupled software components.
  - All communication is through messages.
  - Web application messages always go through clients.
  - The only shared memory is through the session object - which is very restricted.
  - The definition of state is quite different.
- Inherently concurrent and often distributed.
- Most components are relatively small.
- Uses numerous new technologies, often mixed together.
  - JSP, ASP, Java, JavaBeans, Javascript, Ajax, PHP etc.

Various ways of deploying software

- Bundled: Pre-installed on computer.
- Shrink-wrap: Bought and installed by end-users.
- Contract: Purchaser pays developer to develop and install, usually for a fixed price.
- Embedded: Installed on a hardware device, usually with no direct communication with user.
- Web: Executed across the Internet through HTTP.

NPTEL

Web applications: Separation of concerns

Various layers of architecture:

- Presentation layer: HTML, output and UI.
- Data content layer: Computation, data access.
- Data representation layer: In-memory data storage.
- Data storage layer: Permanent data storage.

Testing each layer separately and their integration is also important.

NPTEL

General issues with web applications

- Web applications are heterogeneous, dynamic and must satisfy very high quality attributes.
- Use of the Web is hindered by low quality web sites and applications.

NPTEL

Web applications: Specific problems

- HTTP is a stateless protocol.
  - Each request is independent of previous request.
  - *State* is managed by using *cookies*,, *session objects* etc.
- Servers have little information about where a request comes from.
- Web site software is extremely loosely coupled.
  - Coupled through the Internet - separated by space.
  - Coupled to diverse hardware devices.
  - Written in diverse software languages.
  - New components can be added during execution.
- Users can change the flow of control arbitrarily.
  - Back button, forward button, URL rewriting, refresh.

Our techniques vs. testing web software

- Traditional *graphs* like control flow graphs, call graphs etc. do not apply.
- *State behavior* is hard to model and describe.
- All inputs go through the HTML UI: low *controllability*.
- Hard to get access to server-side state (memory, files, database) – low *observability*.
- Not clear what *logic predicates* can be effectively used.
- No model for *mutation operators* on web software.

NPTEL

Three categories of web applications

The issues for testing web applications can be divided into three broad categories:

- Testing static hyper text web sites.
- Testing dynamic web applications.
- Testing web services.

NPTEL

Categories

- A web page contains HTML content that can be viewed in a single browser window.
  - It may be stored as a static HTML file, or may be dynamically generated by software like JSP, ASP etc.
- A web site is a collection of web pages and associated software elements that are related semantically by content and syntactically through links and other control mechanisms.
- A static web page is unvarying and the same to all users.
  - Usually stored as a HTML file on a server.
- A dynamic web page is created by a program on demand.
  - Its contents and structure maybe dependent on previous inputs from the user, the state on the web server and other inputs like user's location, time etc.

Test case for web applications

A test case for a web application is a sequence of interactions between components on clients and servers.
They are paths of transitions through the web applications.
Generating these paths can be challenging for dynamic web pages.

NPTEL

Testing static hyper text web sites

- This is *not* program testing, but checking that all the HTML connections are valid.
- The main issue to test for is dead links, i.e., links to URLs that are no longer valid.
- We should also evaluate several non-functional parameters:
  - Load testing
  - Performance evaluation
  - Access control issues

NPTEL

Graph models of static web sites

- The usual model is that of a graph.
  - Nodes are web pages.
  - Edges are HTML links.
- The graph is built by starting with an introductory page and recursively doing BFS of all links from that page.
- The graph is then tested for edge coverage: traverse each edge (link) in the graph.

NPTEL

Testing dynamic web applications

Testing dynamic web applications.

- Client-side testing.
- Server-side testing.

NPTEL

Client vs. server side testing

- For web applications, the user interface is on the client and the actual software is on the server.
- Clients and server are separated.
- Tester typically has no access to data, state or the source code on the server.
- We begin with client-side testing strategies.

NPTEL

Client-side (black-box) Testing

- Clients provide inputs to web software residing on a server.
- Inputs: HTML form elements.
  - Text boxes, buttons, dropdown lists, links etc.
- Inputs can be generated or chosen.
  - Techniques for generating inputs:
    - Supplied by the tester,
    - Generated randomly,
    - Generated from user-session data collected from previous users of the software.
  - Techniques for choosing inputs:
    - Bypass testing: Values that violate constraints on the inputs, as defined by client-side information.
- The problem of finding all input screens in a web application is undecidable.

NPTEL

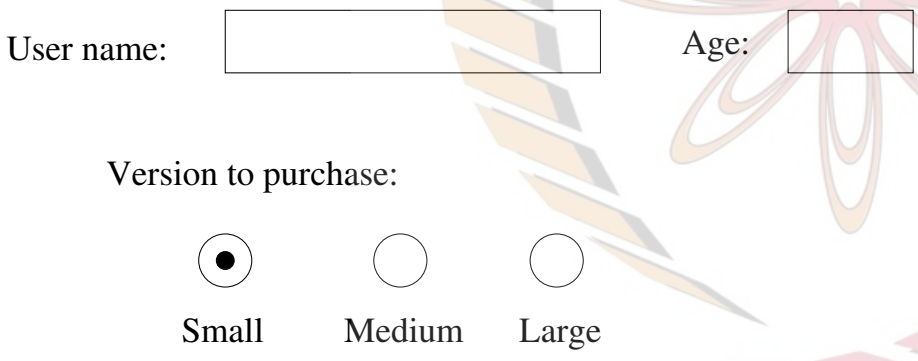
Test value selection

- Challenge: How to automatically provide effective test values?
- Semantic Domain Problem:
  - Values within the application domain are needed.
  - Enumeration of all possible test values is inefficient.
- Possible solutions:
  - Random values (ineffective)
  - Automatically generated values (very hard!)
  - User data (incomplete)
  - Study application and construct a set of values (feasible)
  - Tester-supplied inputs (feasible but expensive)

Bypass testing approach to finding inputs

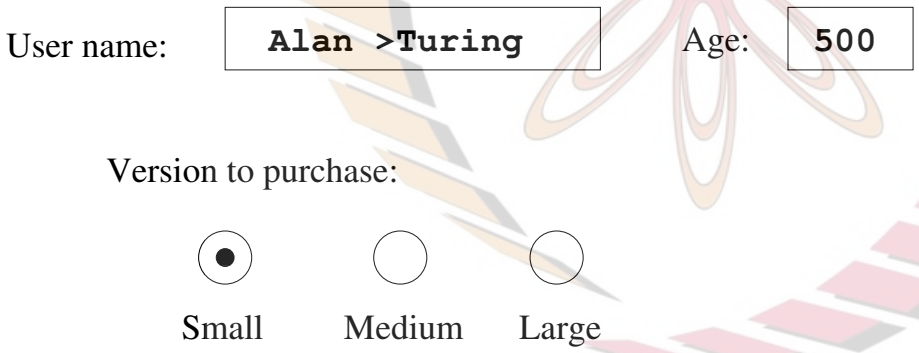
- Web applications impose constraints on inputs through HTML forms.
- Constraints come in two forms:
  - Client-side script validation: Programs run on the client to check the syntax of input data before sending it to server.
  - Uses explicit attributes associated with HTML form fields. E.g., a text box can be set to only allow strings up to a maximum length.
- Bypass testing creates inputs that *intentionally* violate these validation rules and constraints.
  - Created inputs are directly submitted to the web application without letting the web page validate them.

Bypass testing: Example



NPTEL

Bypass testing: Example



- User name should be plain text only.
- Age should be between 18 and 110.

NPTEL

Abbreviated HTML for example

```
<FORM>
   <INPUT Type="text" Name="username" Size=20>
   <INPUT Type="text" Name="age" Size=3 Maxlength=3>
   <P> Version to purchase:

   <INPUT Type="radio" Name="version" Value="150" Checked>
   <INPUT Type="radio" Name="version" Value="250">
   <INPUT Type="radio" Name="version" Value="500">
   <INPUT Type="submit" onClick="return checkInfo(this.form)">
   <INPUT Type="hidden" isLoggedIn="no">
</FORM>
```

Bypass testing

- The basic idea in bypass testing is to let a tester *save* and *modify* the HTML.
- This way, client side checking/validation done routinely is by-passed and the modified data is sent to server.
- It can be used to see if the server crashes on the modified data.
  - Checks for security and robustness.
  - Also checks for common mistakes in inputs.

NPTEL

Bypassing Abbreviated HTML example

```
<FORM>
    <INPUT Type="text" Name="username" Size=20>
    <INPUT Type="text" Name="age" Size=3 ~~Maxlength=3~~>
    <P> Version to purchase:

    <INPUT Type="radio" Name="version" Value="150" Checked>
    <INPUT Type="radio" Name="version" Value="250" >
    <INPUT Type="radio" Name="version" Value=~~"500"~~>
    <INPUT Type="submit" onClick=~~" return checkInfo(this.form)"~~ >
    <INPUT Type="hidden" isLoggedIn=~~"no"~~>
</FORM>
```

Bypass testing: Client vs. server side

- Bypass testing modifies inputs.
- Can be done at the client side or server side.
- Client side inputs are *safer* and *easier* to handle.
- Server side inputs can be modified too, but, can be risky if they *corrupt* data in the server.

NPTEL

Types of client input validation

- Client side input validation is performed by HTML form controls, their attributes and client side scripts that access DOM.
- Validation types are categorized as HTML and scripting.
  - HTML supports syntactic validation.
  - Client scripting can perform both syntactic and semantic validation.

HTML	**Scripting constraints**
Length (max input characters)	Data Type (e.g. integer check)
Value (preset values)	Data Format (e.g. ZIP code format)
Transfer Mode (GET or POST)	Data Value (e.g. age value range)
Field Element (preset fields)	Inter-Value (e.g. credit # + expiry date)
Target URL (links with values)	Invalid Characters (e.g. <,&)

Example client-side constraint rules

- Violate size restrictions on strings
- Introduce values not included in static choices
  - Radio boxes
  - Select (drop-down) lists
- Violate hard-coded values
- Use values that JavaScripts flag as errors
- Change "transfer mode" (get, post, ...)
- Change destination URLs

NPTEL

Example server-slide constraint rules

- Data type conversion
- Data format validation
- Inter-field constraint validation
- Inter-request data fields (cookies, hidden)

NPTEL

Example security violation rules

The following modifications could be done to check for typical security vulnerabilities on web pages:

- Empty String
- Commas: ,
- Single and double quotes: or
- Tag symbols: < and >
- Directory paths: .. ../
- Strings starting with forward slash: /
- Strings starting with a period: .
- Ampersands: &
- Control character: NIL, newline
- Characters with high bit set: 254 and 255

NPTEL

Test value selection

It is a challenge to *automatically* design test values that are effective in terms of identifying faults for dynamic web applications. The following are some of the ideas/techniques known:

- Random values: This is mostly ineffective.
- Taking values from session log files: This is feasible but, can be incomplete.
- Tester giving input based on domain knowledge: This is feasible.

There are papers available in each of the above techniques.

NPTEL

Server responses on the test case

The effectiveness of the test case for a web application can be determined by the response of the server on the test cases.
Broad categorization of server responses:

- Valid responses: Invalid inputs are adequately processed by the server.
  - Server provides an explicit message regarding the violation.
  - Server provides a generic error message.
  - Server ignores the invalid input.
- Faults and failures: Invalid inputs cause abnormal server error.
- Exposure: Invalid inputs are not recognized by the server and abnormal software behavior is exposed to users.
  - Could even result in corruption of data in the server.

Server-side (white-box) testing

- If server-side source code is available, we an use our usual graph models to test the server.
- Control flow graph exhibits only *static* models, not effective for web applications.
- Presentation layer of a web application contains the software and is useful to do testing.
- For software in the presentation layer, two graph models exist:
  - Component Interaction Model (CIM)
  - Application Transition Graph (ATG)

Atomic sections

We need the notion of atomic sections to understand the graph models.

- An atomic section is a section of HTML with the property that if any part of the section is sent to a client, the entire section is.
  - May include JavaScript
  - All or nothing property
- A HTML file is an atomic section.
- Atomic sections may be empty.
- A content variable is a program variable that provides data to an atomic section.

NPTEL

Atomic sections: Example

	PrintWriter out = response.getWriter();
P1 =	out.println ("<HTML>") out.println ("<HEAD><TITLE>"+title+"</TITLE></HEAD>)" out.println ("<BODY>")
	if(User) {
P2 =	out.println ("<CENTER> Welcome!</CENTER>"); for (int i=0; i<myVector.size(); i++) if (myVector.elementAt(i).size > 10)
P3 =	out.println (" "+myVector.elementAt(i)+" ");
	else
P4 =	out.println (" "+myVector.elementAt(i)+" ");
	} else
P5 = P6 =	{ } out.println ("</BODY></HTML>"); out.close ();

Atomic sections are coloured in red. P5 is an empty atomic section. Content variables are coloured green.

Graph models of web applications

- Graph models of server software is restricted to presentation layer only.
- Two abstract graph models are available.
- Component Interaction Model (CIM): As a graph, CIM
  - Models individual components.
  - Combines atomic sections.
  - Intra-component.
- Application Transition Graph (ATG): As a graph, ATG is
  - Each node is one CIM.
  - Edges are transitions among CIMs.
  - Inter-component.

Component Expressions

- Atomic sections are *combined* to model dynamically generated web pages.
- There are four ways to combine and generate component expressions.
  - Sequence: P1 · P2.
  - Selection: P1 | P2.
  - Iteration: $P1^*$.
  - Aggregation: P1 {P2}, P2 is included inside P1.
- The example we just saw gives the component expression P1 · (P2 · (P3 | P4)* | P5) · P6.
- There are algorithms to generate these expressions automatically.

NPTEL

Modelling component transitions

There are five types of transitions/edges.

- Simple Link Transition: An HTML link (<A> tag).
- Form Link Transition: Form submission link.
- Component Expression Transition: Execution of a software component causes a component expression to be sent to the client.
- Operational Transition: A transition out of the softwares control.
  - Back button, Forward button, Refresh button, User edits the URL, Browser reloads from cache
- Redirect Transition: Server side transition, invisible to user.

CIM Example: gradeServlet

	ID = request.getParameter ("Id");
	passWord = request.getParameter ("Password");
	retry = request.getParameter ("Retry");
	PrintWriter out = response.getWriter();

P1 =	out.println ("<HTML><HEAD><TITLE>" +title+ "</TITLE></HEAD><BODY>)"
	if ((Validate (ID, passWord)) {
P2 =	out.println ("<B> Grade Report </B>"); for (int I=0; I < numberOfCourse; I++)
P3 =	out.println(" " +courseName(I)+""+ courseGrade(I)+" "); } else if (retry < 3) {

P4 =	retry++;
	out.println("Wrong ID or wrong password");
	out.println("<FORM Method="get" Action="gradeServlet">);
	out.println("<INPUT Type="text" Name="Id" Size=10>");
	out.println("<INPUT Type="password" Name="Password" Width=20>");
	out.println("<INPUT Type="hidden" Name="Retry" Value="+(retry)+">");
	out.println("<INPUT Type="submit" Name="Submit" Value="submit">");
	out.println("<a href="sendMail">Send mail to the professor<A>");

CIM for gradeServlet

- Start page S is `login.html`.
- A = {P1, P2, P3, P4, P5, P6}.
- CE = gradeServlet = P1 · ((P2 · P3*) | P4 | P5) · P6.
- Transitions T = {login.html → gradeServlet[get,(Id, Password, Retry)], gradeServlet.P4 → sendMail [get, ()], gradeServlet.P4 → gradeServlet [get, (Retry)] }

NPTEL

Application Transition Graph

- Γ: Finite set of web components.
- Θ: Set of transitions among web software components
  - Includes type of HTTP request and data.
- Σ: Set of variables that define the web application state.
- α: Set of start pages.

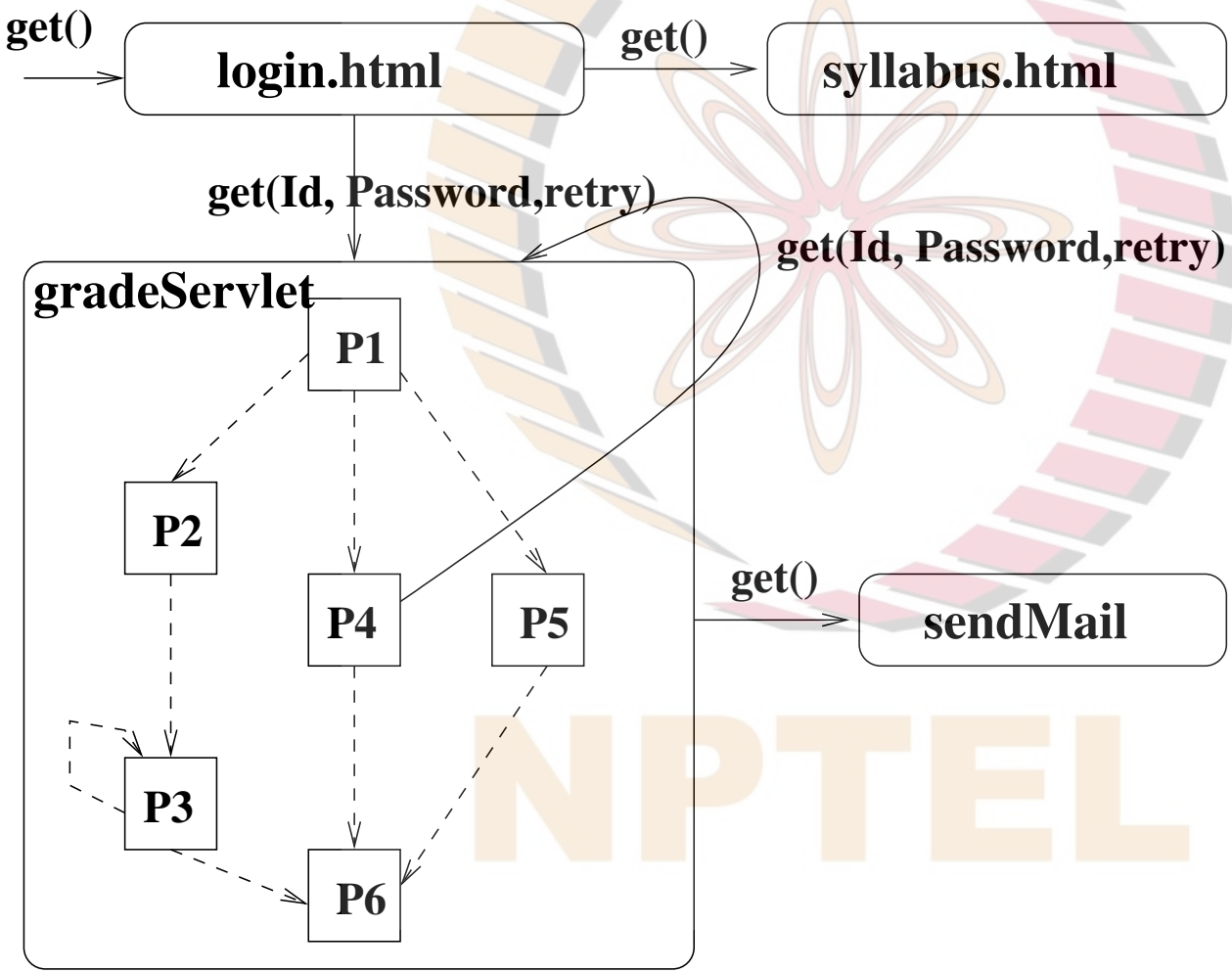
NPTEL

ATG for gradeServelet

- $\Gamma = \{$login.html, gradeServlet, sendMail, syllabus.html $\}$.
- $\Theta = \{$login.html $\rightarrow$ syllabus.html[get,()], login.html $\rightarrow$ gradeServlet [get,(Id,Password,Retry)], gradeServlet.P4 $\rightarrow$ sendMail[get,()], gradeServlet.P4 $\rightarrow$ gradeServlet [get, (Retry)] $\}$
- $\Sigma = \{$Id, Password, Retry$\}$.
- $\alpha = \{$login.html$\}$.

NPTEL

ATG For gradeServelet



Testing based on ATGs and CIMs

- Testing based on atomic sections, in turn, ATGs and CIMs provide a fundamental mechanism to model system level behavior of the presentation layer of web applications.
- Such testing can handle
  - Distributed integration.
  - Dynamically generated HTML web pages.
  - Operational transitions.
- The graphs CIM and ATG need to be generated by manual analysis of software source, can be non-trivial.
- Once generated, they provide a precise model of behavior of web software and is valuable for testing.

Some current open questions/problems

- There is no notion of data-flow based testing that is known for web applications. Static definition+use is difficult to determine.
- ATG cannot be automatically generated.
- Issues involving multiple users, session data, concurrency, dynamic integration etc. cannot be handled.

NPTEL

Website of testing tools on web applications

A comprehensive website on various web testing tools (more than 500 of them!) is maintained by Rick Hower:
`http://www.softwareqatest.com/qatweb1.html`
A good number of these tools do non-functional testing for quality attributes, a topic that we didn't cover in this course.

NPTEL

Conclusion: Web applications testing

- The Web provides a new way to deploy software
- Web applications:
  - offer many advantages
  - use many new technologies
  - introduce fascinating new problems
- Web Software Engineering is just beginning
- Two very useful techniques:
  - Bypass testing: Easy to automate, no source needed
  - Atomic sections: A fundamental model
- This is a very active research area.

Some references

Here are some references for web applications testing.

- Sebastian Elbaum, Gregg Rothermel, Srikanth Karre and Marc Fisher II, Leveraging user-session data to support web application testing, in *IEEE Transactions on Software Engineering*, 31(3), 2005.
- Jeff Offutt, Ye Wu, Xiaochen Du and Hong Huang, Bypass testing of web applications, in *Proc. IEEE ISSRE*, 2004.
- Filippo Ricca and Paolo Tonella, Analysis and testing of web applications, in *Proc. ACM ICSE*, 25-34, 2004.
- Jeff Offutt and Ye Wu, Modeling presentation layer of web applications for testing, *Software and Systems Modelling*, 9(2), 257-280, 2010.